

说明

本应用笔记介绍了如何使用 CH32 系列芯片的 SDIO 接口将其配置为从机设备，以响应外部 SDIO 主机的命令并进行数据传输。为此，文中以 CH32V407 为例，首先概述了 SDIO 控制器特性，然后提供了一个完整的 SDIO 从模式设计实例，包括硬件连接图、寄存器配置方法、命令的响应机制以及数据收发流程。

适用范围

适用范围	系列
通用 MCU	CH32V307 CH32H417 CH32V407

目录

说明	1
目录	2
表格索引	2
图片索引	2
第 1 章 CH32 系列 SDIO 接口概述	1
1.1 概述	1
第 2 章 SDIO 从模式设计实例	2
2.1 硬件连接	2
2.1.1 引脚连接	2
2.2 寄存器配置	2
2.2.1 主机配置	2
2.2.2 从机配置	5
第 3 章 测试与验证	8
历史版本	9
声明	10

表格索引

信号引脚分配	2
--------------	---

图片索引

SDIO 主从通信示意图	1
--------------------	---

第1章 CH32 系列 SDIO 接口概述

1.1 概述

SDIO 接口是指微控制器上一个为操作 SD 卡等外部存储卡或其他设备而设计的通信接口，是微控制器的一个外设。微控制器的 SDIO 直接挂载在 HB 总线上，由 HCLK 直接提供时钟，能实现较高的通讯速度，微控制器的 SDIO 用作 SDIO 主机，被控制的设备也被统称 SDIO 设备或者 SDIO 的从机，本文章主要就是指导用户如何将 MCU 配置成 SDIO 的从机设备，从而和主机设备通信，完成数据快速传输。

CH32 系列作为青稞 RISC-V 生态中的工业级互联型 MCU，其 SDIO 从机控制器并非传统的数据透传通道，而是构建通信系统的关键硬件加速单元。在大数据量通信的嵌入式系统设计中，主机处理器（如应用级 ARM SoC、另一颗 RISC-V MCU 或 FPGA 逻辑阵列）通常需要与 CH32 系列之间以高吞吐量、低延迟、零 CPU 干预的方式交换数据包与实时控制指令，直接在寄存器层与总线级完成数据搬运。配合 CH32 系列的 DMA 控制器，提供的多维度数据传输能力，从机控制器可通过 DMA 和中断协同机制，在无需内核主动参与的情况下，完成从数据接收、搬移到回复的全链路闭环，将物理层传输效率推向接口极限。其硬件设计严格遵循 SDIO 规范，所以可以和多种符合协议的主机设备进行通信。

图 1-1 SDIO 主从通信示意图

SDIO 有 1 位、4 位和 8 位三种数据线宽度可选

第2章 SDIO 从模式设计实例

2.1 硬件连接

SDIO 接口最多需要 10 根信号线组成（8 位模式下使用 DATA0- DATA7 数据线，4 位模式下使用 DATA0- DATA3 数据线，1 位模式仅使用 DATA0），本设计举例 4 位模式。

2.1.1 引脚连接

实例中，将两个 CH32V407 进行连接，一个做主机，一个做从机，进行命令和数据的通信。

表 2-1 信号引脚分配

信号名称	方向	描述	CH32V407 引脚
SDIO_CLK	输入	时钟信号，由主机提供	PC12
SDIO_CMD	双向	命令/响应线	PD2
SDIO_D0	双向	数据线 0	PC8
SDIO_D1	双向	数据线 1	PC9
SDIO_D2	双向	数据线 2	PC10
SDIO_D3	双向	数据线 3	PC11

硬件连接要点：

1. 上拉电阻：CMD 和 DATA 信号线在空闲时应保持高电平，通常需要在主机制板端配置 10kΩ ~ 50kΩ 的上拉电阻，以确保总线空闲时信号线状态稳定。
2. 电平匹配：CH32V407 的 I/O 电平以 VDDIO 电源域为基准。若主机 I/O 电压与 CH32V407 VDDIO（通常为 3.3V）不一致，必须增加电平转换电路。
3. 信号完整性：SDIO 接口在 4 线模式下可达 80MHz 传输速率，高速通信时建议进行阻抗匹配设计，避免使用杜邦线连接，推荐采用 PCB 短距直连方式。

2.2 寄存器配置

主机和从机分别进行配置。SDIO 总线遵循“主机发起、从机响应”的通信原则，所有数据传输操作均由主机通过发送命令启动。

2.2.1 主机配置

主机配置如下：包括初始化 IO 口，时钟，数据位宽，采样边沿等：

```
SD_Init( void )
{
    GPIO_InitTypeDef  GPIO_InitStructure;

    SD_Error errorstatus = SD_OK;

    RCC_PB2PeriphClockCmd(RCC_PB2Periph_GPIOC | RCC_PB2Periph_GPIOD, ENABLE );
    RCC_HBPeriphClockCmd( RCC_HBPeriph_SDIO, ENABLE );
    RCC_PB2PeriphClockCmd( RCC_PB2Periph_AFIO, ENABLE );

    GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_10 | GPIO_Pin_11 | GPIO_Pin_12;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );
}
```

```

    GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_8 | GPIO_Pin_9 ;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOD, &GPIO_InitStructure );

    SDIO_DeInit();

    errorstatus = SD_PowerON();

    return errorstatus;
}

SD_Error SD_PowerON( void )
{
    SD_Error errorstatus = SD_OK;
    SDIO_InitStructure.SDIO_ClockDiv = 0;    //设置分频系数 SDIOCLK=HCLK/ (ClockDiv+2)
    SDIO_InitStructure.SDIO_ClockEdge = SDIO_ClockEdge_Falling; //设置采样边沿
    SDIO_InitStructure.SDIO_ClockBypass = SDIO_ClockBypass_Disable; //是否开启时钟旁路
    SDIO_InitStructure.SDIO_ClockPowerSave = SDIO_ClockPowerSave_Disable; //空闲时时钟关闭使能
    SDIO_InitStructure.SDIO_BusWide = SDIO_BusWide_4b; //数据位宽
    SDIO_InitStructure.SDIO_HardwareFlowControl = SDIO_HardwareFlowControl_Disable; //硬件流
    SDIO_Init( &SDIO_InitStructure );

    SDIO_SetPowerState( SDIO_PowerState_ON ); //设置为上电状态

    SDIO_ClockCmd(Enable ); //开启时钟

    return( errorstatus );
}

```

初始化完成后，主机开始发送 CMD，提示从机已经做好准备，等待从机回复。从机只支持 R1 类型的应答：

```

while ((SDIO->STA&SDIO_FLAG_CMDREND)==0)
{
    SDIO_CmdInitStructure.SDIO_Argument = 0x6; //发送的 ARG
    SDIO_CmdInitStructure.SDIO_CmdIndex = 0x05; //发送的 CMD
    SDIO_CmdInitStructure.SDIO_Response = SDIO_Response_Short; //等待响应类型
    SDIO_CmdInitStructure.SDIO_Wait = SDIO_Wait_No;
    SDIO_CmdInitStructure.SDIO_CPSM = SDIO_CPSM_Enable; //使能发送
    SDIO_SendCommand( &SDIO_CmdInitStructure );
}

```

```

}
```

当状态机改变，主机状态变为 CMDREND:，已经收到响应，CRC 检验成功；主机开始发送数据

```

//DMA 配置函数
```

```

void SD_DMA_Config( u32 *mbuf, u32 bufsize, u32 DMA_DIR )
{
    DMA_InitTypeDef DMA_InitStructure;

    RCC_HBPeriphClockCmd( RCC_HBPeriph_DMA2, ENABLE );

    DMA_DeInit( DMA2_Channel4 );
    DMA_Cmd( DMA2_Channel4, DISABLE );

    DMA_InitStructure.DMA_PeripheralBaseAddr = ( u32 )&SDIO->FIFO;
    DMA_InitStructure.DMA_MemoryBaseAddr = ( u32 )mbuf;
    DMA_InitStructure.DMA_DIR = DMA_DIR;
    DMA_InitStructure.DMA_BufferSize = bufsize/4 ;
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
    DMA_InitStructure.DMA_Priority = DMA_Priority_High;
    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
    DMA_Init( DMA2_Channel4, &DMA_InitStructure );
}

```

```

//主机发送函数
```

```

Void HOST_Send()
{

```

```

    SD_DMA_Config(( u32 * )buf,512,DMA_DIR_PeripheralDST); //调用 DMA 配置传参

```

```

    SDIO_DataInitStructure.SDIO_DataBlockSize = 9 << 4; ; //传输数据块长度：512B
    SDIO_DataInitStructure.SDIO_DataLength = 512 ;
    SDIO_DataInitStructure.SDIO_DataTimeOut = SD_DATATIMEOUT ; //设置超时时间
    SDIO_DataInitStructure.SDIO_DPSM = SDIO_DPSM_Enable;
    SDIO_DataInitStructure.SDIO_TransferDir = SDIO_TransferDir_ToCard; //传输方向
    SDIO_DataInitStructure.SDIO_TransferMode = SDIO_TransferMode_Block; //块模式传输

```

```

    SDIO_DataConfig( &SDIO_DataInitStructure );

```

```

    SDIO_DMACmd( ENABLE ); //使能 DMA
    DMA_Cmd( DMA2_Channel4, ENABLE );

    while( ( ( DMA2->INTFR & 0X2000 ) == RESET ) ) //等待 DMA 传输完成标志
    {
    }

    while(((SDIO->STA&SDIO_FLAG_DBCKEND)==0)) //等待已经发送或者接受了数据块且 CRC 校验通过的标志。
    {
    }

    SDIO->DCTRL&=~(1<<0); //关闭 SDIO 发送

```

```

SDIO_DMACmd( DISABLE );

DMA_Cmd( DMA2_Channel4, DISABLE );//关闭 DMA

printf("data send OK!!!\r\n");

SDIO->ICR=0xFFFFF; //清除 SDIO 状态位
}

```

2.2.2 从机配置

从机配置如下：包括初始化 IO 口，数据位宽等

```

SD_Init( void )
{
    GPIO_InitTypeDef GPIO_InitStructure;
    SD_Error errorstatus = SD_OK;

    RCC_PB2PeriphClockCmd(RCC_PB2Periph_GPIOC | RCC_PB2Periph_GPIOD, ENABLE );
    RCC_HBPeriphClockCmd( RCC_HBPeriph_SDIO, ENABLE );
    RCC_PB2PeriphClockCmd( RCC_PB2Periph_AFIO, ENABLE );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10 | GPIO_Pin_11 | GPIO_Pin_12;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8 | GPIO_Pin_9 ;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOC, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init( GPIOD, &GPIO_InitStructure );

    SDIO_DeInit();

    errorstatus = SD_PowerON();

    return errorstatus;
}

SD_Error SD_PowerON( void )
{
    SD_Error errorstatus = SD_OK;
    SDIO_InitStructure.SDIO_ClockDiv = 0; //设置分频系数 SDIOCLK=HCLK/(ClockDiv+2)
    SDIO_InitStructure.SDIO_ClockEdge = SDIO_ClockEdge_Falling; //设置采样边沿
    SDIO_InitStructure.SDIO_ClockBypass = SDIO_ClockBypass_Disable; //是否开启时钟旁路
    SDIO_InitStructure.SDIO_ClockPowerSave = SDIO_ClockPowerSave_Disable; //空闲时时钟关闭使能
    SDIO_InitStructure.SDIO_BusWide = SDIO_BusWide_4b; //数据位宽
    SDIO_InitStructure.SDIO_HardwareFlowControl = SDIO_HardwareFlowControl_Disable; //硬件流
    SDIO_Init( &SDIO_InitStructure );

    SDIO_SetPowerState( SDIO_PowerState_ON ); //设置为上电状态

    SDIO_ClockCmd(Enable ); //开启时钟

    SDIO->DCTRL2|=(1<<24); ///SLV 模式开启
    return( errorstatus );
}

```

初始化完成后，从机配置 CMD 的 R1 回复，方便在收到主机的 CMD 后自动回复，以提示主机已经做好准备，并进入等待主机发送 CMD 状态。

```

SDIO->ARG=0x13;//////////R1 回复主机 0x13;

while ((SDIO->STA&SDIO_FLAG_CMDSENT)==0)
{

```

```
}

```

当状态机改变，从机状态变为 CMDREND:，已经收到响应；从机开始接收数据。

```
//DMA 配置函数

```

```
void SD_DMA_Config( u32 *mbuf, u32 bufsize, u32 DMA_DIR )

```

```
{

```

```
    DMA_InitTypeDef DMA_InitStructure;

```

```
    RCC_HBPeriphClockCmd( RCC_HBPeriph_DMA2, ENABLE );

```

```
    DMA_DeInit( DMA2_Channel4 );

```

```
    DMA_Cmd( DMA2_Channel4, DISABLE );

```

```
    DMA_InitStructure.DMA_PeripheralBaseAddr = ( u32 )&SDIO->FIFO;

```

```
    DMA_InitStructure.DMA_MemoryBaseAddr = ( u32 )mbuf;

```

```
    DMA_InitStructure.DMA_DIR = DMA_DIR;

```

```
    DMA_InitStructure.DMA_BufferSize = bufsize/4 ;

```

```
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;

```

```
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;

```

```
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;

```

```
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;

```

```
    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;

```

```
    DMA_InitStructure.DMA_Priority = DMA_Priority_High;

```

```
    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;

```

```
    DMA_Init( DMA2_Channel4, &DMA_InitStructure );

```

```
}

```

```
//主机发送函数

```

```
Void SLV_Rec()

```

```
{

```

```
    SDIO_DataInitStructure.SDIO_DataBlockSize = 9 << 4; ; //传输数据块长度: 512B

```

```
    SDIO_DataInitStructure.SDIO_DataLength = 512 ;

```

```
    SDIO_DataInitStructure.SDIO_DataTimeOut = SD_DATATIMEOUT ;

```

```
    SDIO_DataInitStructure.SDIO_DPSM = SDIO_DPSM_Enable;

```

```
    SDIO_DataInitStructure.SDIO_TransferDir = SDIO_TransferDir_ToSDIO; //传输方向为接收

```

```
    SDIO_DataInitStructure.SDIO_TransferMode = SDIO_TransferMode_Block; //块模式传输

```

```
SDIO_DataConfig( &SDIO_DataInitStructure );

```

```
SD_DMA_Config(( u32 * )Readbuf, 512, DMA_DIR_PeripheralSRC);

```

```
    SDIO_DMACmd( ENABLE ); //使能 DMA

```

```
    DMA_Cmd( DMA2_Channel4, ENABLE );

```

```
    while( ( ( DMA2->INTFR & 0X2000 ) == RESET ) ) //等待 DMA 传输完成标志

```

```
    {

```

```
    }

```

```
    while(((SDIO->STA&SDIO_FLAG_DBCKEND)==0)) //等待已经发送或者接受了数据块且 CRC 校验通过的标志。

```

```
    {

```

```
    }

```



```
SDIO->DCTRL&=~(1<<0); //关闭 SDIO 接收
SDIO_DMACmd( DISABLE );
DMA_Cmd( DMA2_Channel4, DISABLE );//关闭 DMA
printf("data send OK!!!\r\n");
SDIO->ICR=0xFFFFFFFF; //清除 SDIO 状态位
}
```

完成上述配置后，可以完成一轮的主机发送，从机接收的操作。

第3章 测试与验证

完成上述配置后，建议按照以下步骤分阶段验证 SDIO 从机功能：

1. 硬件连通性测试：使用示波器测量 CMD 线和 DATA 线上是否有信号波形，确认主机与从机的物理连接正常。
2. 命令识别测试：从机在检测到 SDIO_FLAG_CMDSENT 后，打印接收到的命令参数会放在 RESP1，接收到的命令索引会放在 RESP2，查看 CMD 发送是否正确，或者使用逻辑分析仪进行抓取解码。
3. 批量数据收发测试：从主机端发送，从机通过串口打印接收到的数据内容，验证接收功能。查看主机和从机的 STA，是否有异常报错。

历史版本

日期	版本	变更内容
2026/6/25	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。